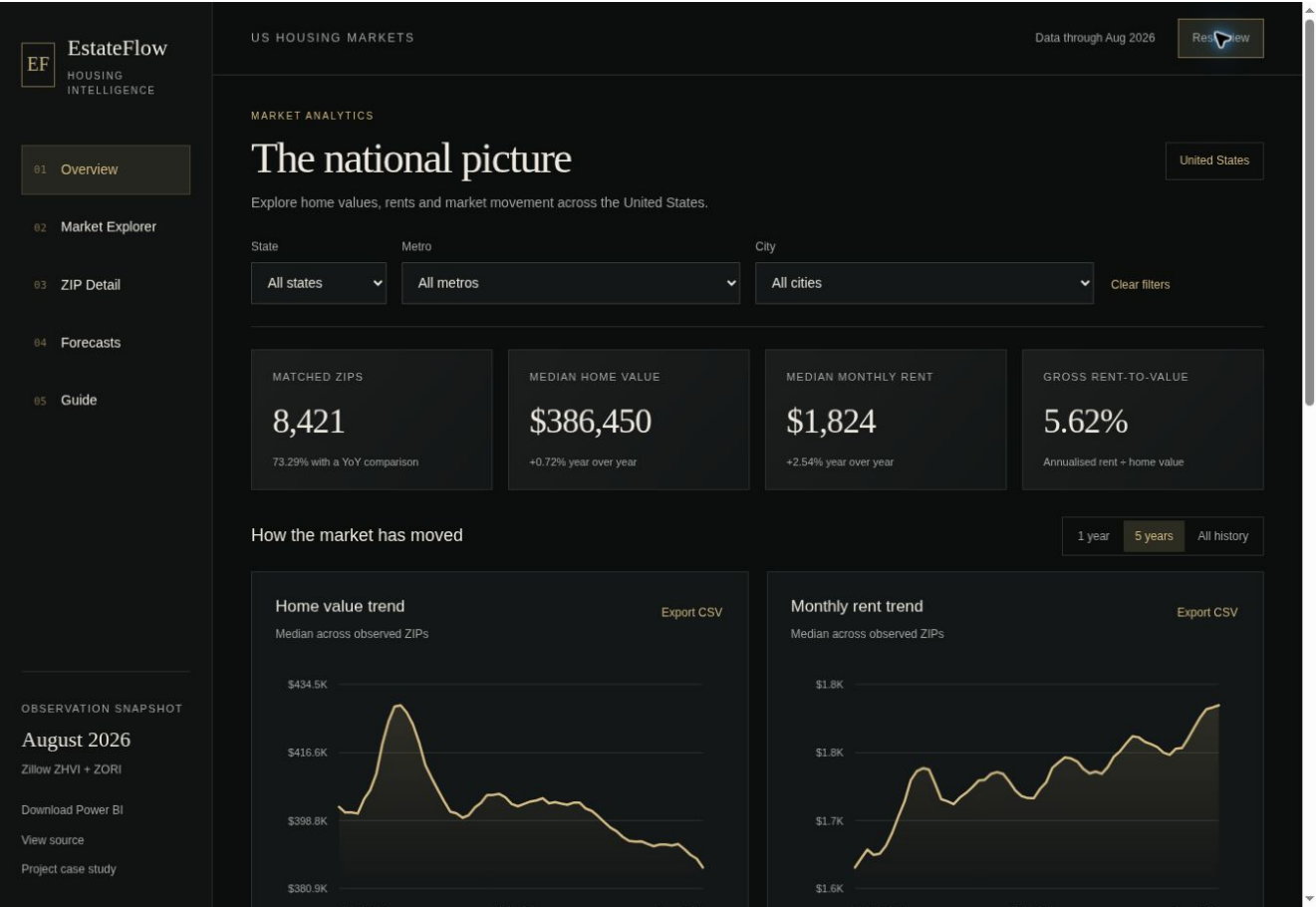


EstateFlow

Project handbook

Architecture, operating procedures, methodology
and development record



Built by Yuvraj Riyar
Release edition 1.0 | 1 October 2026

OBSERVATION SNAPSHOT: 31 AUGUST 2026

An evidence-based guide to the shipped analytical pipeline, Power BI report and public web dashboard. Fixed snapshot, not a real-time housing service.

Contents

01 | Executive brief

02 | Scope, users and decision workflow

03 | Source data and vintage control

04 | Architecture and repository ownership

05 | Python ingestion and execution

06 | SQL models and grain

07 | Metric dictionary and interpretation

08 | Data quality and verification strategy

09 | Forecast experiment: design

10 | Forecast results and responsible claims

11 | Power BI implementation and use

12 | Web dashboard implementation

13 | First-run pipeline operating guide

14 | Refresh, export and release workflow

15 | Troubleshooting and safe recovery

16 | Development journey and design decisions

17 | AI-assisted work, ownership and interview explanation

18 | Maintenance, future ideas and release checklist

19 | Provenance and document maintenance

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

Use chapters 01-04 for orientation, 05-12 for implementation, 13-15 for operations, and 16-19 for decisions, maintenance and evidence.

Source of truth: inspected code and measured release evidence. Historical recollections and proposed improvements are labelled. No claims of investment-grade forecasting or enterprise production readiness.

01 | Executive brief

EstateFlow turns wide Zillow housing files into a reproducible ZIP-month analytical model, then makes those results available through a public browser dashboard and a downloadable five-page Power BI report. Its value is the complete path from source data to usable comparison, with explicit checks and visible limitations.

The business question is straightforward: how do typical home values, rents and market movement compare across US ZIP codes? A reader can begin with national medians, narrow by state, metro or city, inspect a ZIP's history, and examine a separately labelled forecasting experiment.

This is a solo portfolio project. The implemented sources are Zillow ZHVI and ZORI. FRED and Census ACS are potential extensions, not inputs to the released metrics. The core grain is ZIP-month, not metro-month. Metro and state views aggregate the ZIP-level data.

Released access routes

Route	What the visitor needs	What it provides
Web dashboard	A browser; no account	Five interactive views, filters, histories, comparisons, forecasts and CSV exports
Power BI download	Free Power BI Desktop on Windows	Imported August snapshot in a self-contained PBIX; a local editable copy
Source project	Python, uv, Docker and PostgreSQL; Desktop for BI editing	Rebuildable pipeline, SQL, tests, forecast experiment and PBIP definitions

Live dashboard: <https://yuvrajriyar.vercel.app/projects/estateflow/dashboard>

Project case study: <https://yuvrajriyar.vercel.app/projects/estateflow>

Pipeline repository: <https://github.com/yuvrajriyar/EstateFlow>

Web repository: <https://github.com/yuvrajriyar/portfolio-website>

What completion means

The software release is shipped. The August pipeline passed its quality gates, forecasts were published, all five Power BI pages were locally refreshed and reviewed, and the web deployment was checked publicly. Completion does not mean perpetual freshness, guaranteed forecast accuracy, exhaustive security certification or a production service-level agreement. Updating and re-evaluating the snapshot is ongoing maintenance.

02 | Scope, users and decision workflow

The primary reader is someone comparing housing markets or reviewing the author's analytical and engineering work. EstateFlow supports screening and investigation. It does not recommend purchasing a property, estimate net cash flow or promise a return.

A useful reading sequence

1. Start with Overview. Read the snapshot date, number of matched ZIPs and YoY coverage before interpreting growth.
2. Filter geographically. Compare home value, monthly rent and gross rent-to-value together rather than selecting the highest ratio alone.
3. Use Market Explorer for broader group comparisons. Examine whether rent momentum and home-value movement differ.
4. Open ZIP Detail. Check observed history and geographic benchmarks for the selected ZIP.
5. Open Forecasts only after understanding the observed data. Read the model and uncertainty labels.
6. Consult Guide for definitions and limits. Export a relevant table if further analysis is needed.

Questions the release can answer

- What are the median home value and monthly rent among the matched ZIPs in a chosen geography?
- How have observed values moved over time, and how many ZIPs support each month's comparison?
- Which ZIPs differ from their metro, state or national peers?
- How much of the latest snapshot supports an exact twelve-month growth calculation?
- For eligible ZIPs, what do validated simple index models project at three, six and twelve months?

Questions outside the release

EstateFlow cannot determine an individual home's appraisal, achievable lease rent, occupancy, operating costs, financing terms, net yield or suitability for a buyer. It does not identify causal drivers of price changes. It is not comprehensive coverage of every US ZIP or property.

03 | Source data and vintage control

ZHVI is the Zillow Home Value Index, a modelled measure of typical home value. The released file covers the middle-tier single-family and condominium series. ZORI is the Zillow Observed Rent Index, based on advertised rents and methodology intended to represent the rental stock. Neither is a complete ledger of realised transactions or signed leases.

The files arrive in wide format: one row per ZIP, nine metadata columns and monthly date columns. Python reshapes non-missing observations into one ZIP-month record. ZIP identifiers stay as five-character strings so 01002 does not become 1002.

Source or output	Records	Distinct ZIPs
ZHVI staging	6,482,537	26,268
ZORI staging	463,419	8,459
Matched historical mart	462,410	8,424
Latest matched August snapshot	8,421	8,421
Experimental forecasts	31,566	5,261

ZHVI begins in January 2000; the joined rent/home history begins in January 2015. The released sources end on 31 August 2026. They were retrieved on 1 October 2026 at 20:28 UTC and had a reported Zillow modification date of 16 September 2026. The source manifest records URLs, dates, observation counts and uncompressed CSV hashes.

Three dates to distinguish

- Observation date: the month represented by a value, here August 2026.
- Retrieval date: when the file was downloaded, here October 2026.
- Release date: when the usable product was published, also October 2026.

Zillow revises past values and coverage. A new download can change historical results, not merely add a month. Therefore, compare source vintages deliberately and retain the manifest. A file from October is not automatically October observations.

Source updater behaviour

`src/estateflow/update_sources.py` downloads both files into a temporary directory, validates their metadata header, dates, ZIP grain, non-empty latest month and positive finite observations, and refuses an older observation vintage. It requires the two latest source months to agree before replacement. ZHVI is saved as gzip; ZORI remains CSV.

Validation precedes replacement, but replacement of two files and the manifest is sequential, not one filesystem transaction. If the process is interrupted during replacement, inspect both files and the manifest before rebuilding. Equal latest months alone do not establish identical retrieval history; hashes provide the stronger identity check.

04 | Architecture and repository ownership

There are two repositories with separate responsibilities. EstateFlow owns the analytical pipeline and Power BI source. The portfolio repository owns the Next.js web dashboard, static exports and public route. The web app is not an embedded Power BI Service report.

Component	Responsibility	Main location
Source updater	Download and validate a matching vintage	EstateFlow: src/estateflow/update_sources.py
Transformation	Reshape, validate and load observed rows	EstateFlow: transform_zori.py; transform_zhvi.py
Staging	Typed source records	EstateFlow: sql/staging/
Intermediate	Inner join on ZIP and month	EstateFlow: sql/intermediate/
Analytical marts	Historical metrics and common-month snapshot	EstateFlow: sql/marts/
Quality gates	Reconciliation and calculation validation	EstateFlow: sql/quality/
Forecast experiment	Evaluation, intervals and explicit publication	EstateFlow: forecasting.py; run_forecast.py
Power BI	Semantic model and report definitions	EstateFlow: powerbi/EstateFlow_PowerBI_Analytics/
Web exporter	Reproduce the reviewed analytical snapshot	Portfolio: scripts/export-estateflow.py
Browser interface	Five views and client interaction	Portfolio: app/projects/estateflow/dashboard/
Public static data	Latest summary, state histories and forecasts	Portfolio: public/data/estateflow/

End-to-end sequence

1. Retrieve and inspect the Zillow source vintage.
2. Load each source into its staging table.
3. Join observed ZIP-month pairs in the intermediate view.
4. Derive historical metrics and the latest common-month snapshot.
5. Run SQL quality gates; stop and investigate any failure.
6. Run the separate forecast evaluation and explicitly publish its table.
7. Refresh Power BI and review the five pages.
8. Export reconciled static web data, build and test the browser app.
9. Publish source changes and deploy; check visitor access after deployment.

The public web path is static: the browser reads JSON files, not the author's PostgreSQL database. This avoids exposing database credentials and makes the released snapshot usable without a running backend.

05 | Python ingestion and execution

`profile_zori.py` and `profile_zhvi.py` support source inspection. The transformations perform the actual reshape and load. Run scripts from the repository root because several paths are relative.

ZORI transformation

The smaller rent dataset is read into pandas with `RegionName` explicitly typed as text. The first nine columns are treated as metadata and the remaining columns as months. A melt converts the file to long form. Missing rent observations are removed rather than estimated. Dates are parsed, fields renamed and columns aligned with the staging schema.

Assertions check positive rent, required values, five-digit ZIPs, ZIP-month uniqueness and matching state columns. The load uses `psycopg COPY` and row-count reconciliation. The table is truncated and replaced within that script's database transaction.

ZHVI transformation

The larger home-value file is read in chunks of 500 source rows. Each chunk follows the same melt, validation and renaming pattern. Chunking limits the size of each expanded pandas frame. `COPY` loads the transformed records, with a cumulative count reconciled against PostgreSQL.

Pipeline runner

`run_pipeline.py` executes staging DDL, both transformation scripts, three model SQL files and five observed-data quality SQL files. Python subprocesses use the current interpreter. SQL runs through Docker Compose `psql` with `ON_ERROR_STOP=1`. Checked subprocess failures stop the runner.

`--skip-` loads rebuilds the views and runs checks against the data already in PostgreSQL. It does not download or reload a newer source. The forecast runner is intentionally separate.

Transaction boundary and practical limits

Each source load is transactional, but the whole multi-script pipeline is not one transaction. ZORI can commit before ZHVI fails. Views are created before the final quality gates run. A failed run must not be treated as a released snapshot, and consumers should not refresh while rebuilding. The controls are fail-fast release gates, not physical isolation that prevents every concurrent reader from querying unapproved data.

Some transformation and exporter checks use Python `assert` statements. Do not run these scripts with Python optimisation flags that disable assertions. Replacing critical assertions with explicit exceptions is a sensible hardening task.

06 | SQL models and grain

Object	Grain	Role
staging.zhvi_zip_month	One observed home-value ZIP-month	Typed home-value source
staging.zori_zip_month	One observed rent ZIP-month	Typed rent source
intermediate.zip_month_housing	One matched ZIP-month	Both indices present for the same ZIP and month
marts.zip_month_market_metrics	One matched ZIP-month	Derived metrics and exact prior-year comparison
marts.latest_zip_market_metrics	One ZIP at the maximum joined month	Consistent latest snapshot
marts.zip_market_forecasts	ZIP, index, horizon and origin snapshot	Experimental estimates and bounds
marts.zip_market_forecast_display	Forecast display records	Reader-friendly labels for Power BI

The intermediate view performs an INNER JOIN on ZIP code and month. It uses geography from the ZHVI row. A ZIP with rent but no matching home value is absent from this model. Smaller matched coverage is an intentional consequence of comparable observations, not evidence that all excluded source records are invalid.

The historical mart LEFT JOINS the same ZIP exactly twelve calendar months earlier. Month-end arithmetic handles February and leap years. It does not use the twelfth previous available row, which would be wrong when months are missing.

The latest mart filters the historical mart to its global maximum month. It does not select each ZIP's independently latest record. All 8,421 latest ZIPs therefore refer to August 2026 rather than a mixture of fresh and stale months.

Example read-only checks

```
SELECT COUNT(*), COUNT(DISTINCT zip_code), MIN(month), MAX(month)
FROM marts.zip_month_market_metrics;
```

```
SELECT COUNT(*), COUNT(DISTINCT month), MIN(month), MAX(month)
FROM marts.latest_zip_market_metrics;
```

```
SELECT zip_code, month, zhvi_usd, zori_usd,
       gross_rent_to_value_pct, home_value_yoy_pct
FROM marts.latest_zip_market_metrics
WHERE zip_code = '01002';
```

For the released vintage, the first query returns 462,410 rows and 8,424 ZIPs; the second returns 8,421 rows and one month. Counts are release-specific acceptance evidence, not universal invariants for future source files.

07 | Metric dictionary and interpretation

Metric	Definition	Important qualification
Home value	ZHVI in USD	Modelled typical value, not a property appraisal
Monthly rent	ZORI in USD	Index of typical observed rent, not contracted income
Annualised rent	Monthly rent multiplied by 12	Run rate at that monthly level, not next-year collected rent
Gross rent-to-value %	Monthly rent $\times$ 12 / home value $\times$ 100	Screening ratio, not net yield or cap rate
Home YoY %	(Current home / exact prior-year home - 1) $\times$ 100	Blank if the matched comparison is missing
Rent YoY %	(Current rent / exact prior-year rent - 1) $\times$ 100	Same exact-calendar comparison requirement
YoY coverage %	ZIPs with a valid comparison / latest filtered ZIPs $\times$ 100	Coverage, not a growth measure

Aggregation rules

Web headline measures are medians across individual matched ZIPs with equal ZIP weight. They are not population-weighted US indices. Home-growth and rent-growth medians use available comparisons; missing values are not converted to zero.

Calculate each ZIP's gross percentage before taking the median. The median of ZIP ratios need not equal median rent $\times$ 12 / median home value. Similarly, do not calculate a national median from state medians. Different monthly coverage can move a historical aggregate even when the market itself changes less.

Released national measures

The latest snapshot contains 8,421 matched ZIPs. Median home value is \$386,450, median monthly rent is \$1,824 and median gross rent-to-value is 5.62%.

Median home YoY is +0.72%; median rent YoY is +2.54%. Exact prior-year comparison coverage is 73.29%. These national web measures are equal-weighted ZIP medians, not population-weighted indices.

Worked example

If a ZIP's home index is \$400,000 and its rent index is \$2,000, annualised rent is \$24,000 and gross rent-to-value is 6%. This says nothing about financing, vacancy, maintenance, management, insurance, property taxes or transaction costs. It cannot be relabelled as a 6% profit or net return.

08 | Data quality and verification strategy

Validation exists at several levels: source-file inspection, Python transformation assertions, database checks, forecast validation, web-export reconciliation and user-interface acceptance. A green build alone does not establish analytical correctness.

Control	What it protects	Expected release result
Required values	ZIP, month and core measures exist	Zero violations
Positive values	Non-positive rents/home values do not pass	Zero violations
Duplicate grain	One row per ZIP-month or latest ZIP	Zero duplicates
Load reconciliation	PostgreSQL count matches transformed rows	Counts agree
Model reconciliation	Mart row count matches the joined model	462,410 in both
Derived calculations	Annualisation and gross ratio independently agree	Zero mismatches
YoY checks	Flags and exact-year calculations agree	Zero mismatches
Latest snapshot	Correct count and one common month	8,421; August 2026
Forecast controls	Unique horizons, positive ordered bounds and counts	Six rows per eligible ZIP

SQL gate files raise exceptions rather than merely printing a failed count. The pipeline runner stops when those exceptions occur. A database NOTICE saying a schema or table already exists is normally informational; an ERROR or a non-zero script exit needs investigation.

Automated tests

`tests/test_forecasting.py` exercises forecast behaviour. `tests/test_forecast_pbir.py` checks authored report structure. `tests/test_quality_gates_postgres.py` uses an actual disposable PostgreSQL database to test valid fixtures and failing observed-data gates.

The integration suite is skipped when `ESTATEFLOW_TEST_DSN` is absent. A local unit-test pass with skipped integration tests is not a PostgreSQL pass. The GitHub Actions workflow provisions PostgreSQL 17 and supplies a test DSN. This handbook inspected that configuration; it does not claim that every historical workflow run succeeded.

WARNING: the integration suite drops staging, intermediate and marts schemas in the database addressed by its DSN. Never point it at your working project database or any database containing valuable data.

09 | Forecast experiment: design

Forecasting is a separate experiment, not part of the observed-data ETL runner. It estimates future Zillow index levels for two metrics and three horizons. Eligibility requires 24 consecutive matched monthly observations ending at the forecast origin. Gaps are not filled.

Candidate models

Flat baseline: carry the latest observed index level forward unchanged. This is a deliberately strong reference for smooth housing indices.

Damped log trend: fit an ordinary least-squares line to the log index over 24 months. Anchor the extrapolation at the latest observation, cap the monthly log slope at plus or minus 0.02, and apply damping of 0.85 raised to each future month. Exponentiate to return to USD.

Selection and validation

For each metric/horizon combination, compare mean absolute log errors in the selection period. The trend needs more than a 1% improvement to be initially selected. A separate validation period then requires more than a 10% improvement in median absolute percentage error over the flat baseline; otherwise the baseline is retained. Selection is pooled by metric/horizon, not a separately tuned model for every ZIP.

Chronological windows for the August vintage

Stage	Target-month window	Purpose
Selection	September 2021 to August 2022	Choose an initial candidate
Validation	September 2022 to August 2023	Require material improvement
Calibration	September 2023 to August 2024	Estimate interval radii
Final retrospective evaluation	September 2025 to August 2026	Measure later performance

A target at month t and horizon h uses an origin at $t-h$. Each prediction uses only the preceding 24 months available through that origin in the loaded series. The latest calibration target precedes the earliest final-evaluation origin even for the twelve-month horizon.

Empirical intervals

Absolute log errors are pooled by metric and horizon in the calibration period. A finite-sample empirical rank supplies an 80% and 95% radius. Add and subtract the radius around the log point forecast, then exponentiate. Bounds are positive and ordered.

These are empirical prediction intervals, not confidence intervals for a mean and not guaranteed coverage. Repeated ZIP-month cases overlap, ZIPs are dependent, market regimes shift and pooled intervals may poorly represent unusual ZIPs.

10 | Forecast results and responsible claims

The August run published 31,566 rows for 5,261 eligible ZIPs: two indices × three horizons per ZIP. Forward target months are November 2026, February 2027 and August 2027. These outcomes have not yet been observed at release.

Index / horizon	Model	Cases	Median error	Flat error	80% coverage	95% coverage
Home / 3m	Flat	58,805	0.65%	0.65%	92.2%	98.5%
Home / 6m	Flat	57,048	1.18%	1.18%	92.8%	98.5%
Home / 12m	Flat	53,111	2.24%	2.24%	92.6%	98.4%
Rent / 3m	Flat	58,805	1.46%	1.46%	77.8%	92.7%
Rent / 6m	Flat	57,048	1.86%	1.86%	81.4%	94.3%
Rent / 12m	Damped trend	53,111	1.74%	2.41%	88.6%	97.6%

Only twelve-month rent forecasting justified replacing the flat baseline. Its median retrospective error fell from 2.41% to 1.74%, a reduction of about 28%. This is specific to the evaluated August vintage, not a claim of universal superiority.

Coverage is visibly different from the nominal labels. The three-month rent 80% interval covered 77.8% of cases, while the home intervals were conservative in this retrospective period. Reporting both point errors and realised coverage is more useful than advertising a single accuracy percentage.

Limitations to retain in every explanation

- Zillow history is revised. Using today's revised series is not a vintage-by-vintage reconstruction of information available at every past date.
- Developers inspected historical outcomes during experiment refinement. The later test period is separated in code but is not a pristine external prospective trial.
- Complete-history eligibility makes forecast coverage non-representative of all ZIPs.
- Models use index history, not local supply, rates, policy or detailed seasonality drivers.
- Errors describe index levels, not individual properties or collected rental income.

The repository's July forecast-methodology results are a historical experiment. Use docs/current-data.md and this August release table for the shipped vintage. Do not silently combine July evaluation results with August forward estimates.

11 | Power BI implementation and use

The editable PBIP contains report definitions and a semantic model. It is designed for source control and excludes the local imported-data cache. The released PBIX includes the reviewed imported August data, so visitors can explore without rebuilding PostgreSQL.

Page	Main use	Acceptance focus
National Housing Market	Headline medians, trends and ZIP screening	Geographic filters and YoY coverage
Market Explorer	State/metro movement and ratio comparisons	Cross-filter behaviour and aggregation
ZIP Detail	Selected ZIP history and benchmarks	One ZIP, correct peer context
Forecast Experiment	Separate index estimates and empirical bounds	One ZIP, two charts and six records
How to Read the Dashboard	Definitions and limits	Observed versus experimental distinction

Visitor instructions

Download EstateFlow_Dashboard.pbix from the v1.0.0 GitHub release and open it in Power BI Desktop for Windows. Do not refresh unless your own database is configured. In Desktop editing mode, hold Ctrl when using navigation buttons. Select one ZIP on Forecast Experiment to populate its charts.

The download is an independent local copy. Editing it does not change the released original. It still exposes Desktop authoring tools; it is not a locked application. The public web app is the browser-first consumption route.

Rebuilder instructions

Run the observed pipeline and publish forecasts, then open `powerbi/EstateFlow_PowerBI_Analytics/EstateFlow_Dashboard.pbip`. Configure the PostgreSQL connection and credentials when prompted. The source model expects `localhost:5432` and database `estateflow`; adjust the model's connection if using different settings.

A Power BI instance in a different VM or computer must connect to the database host, not blindly use `localhost`. A healthy container on the Mac does not guarantee that a Windows Desktop session can reach it. Keep access limited to the intended local environment; never expose PostgreSQL broadly merely to make a refresh work.

Reset and acceptance

The author confirmed navigation, Clear filters and reopening through `Open EstateFlow.cmd` on 1 October 2026. The launcher-based reset is a verified source-project workflow, not a promise that every arbitrary copied PBIX will discard saved filters automatically. Before saving a distributable PBIX, restore the intended opening page and clear filters.

The author reviewed all five refreshed pages. Forecast acceptance included ZIP 01002 with both charts and all six rows; ZIP Detail included San Francisco 94112.

12 | Web dashboard implementation

The web version runs inside the Next.js portfolio on Vercel. The inspected package configuration uses Next.js 16.2.6, React 19.2.6 and TypeScript 5.9.3. It is a distinct React implementation of the analytical experience, not a Power BI embedding workaround.

The interface has five views: Overview, Market Explorer, ZIP Detail, Forecasts and Guide. It uses a charcoal, cream and muted-gold visual system, custom SVG charts, compact tables and responsive navigation. The aim is an application-like experience without Desktop editing tools.

Main frontend files

File	Responsibility
page.tsx	Route entry and reviewed summary data
Dashboard.tsx	View selection, filters, ZIP selection and export interaction
Charts.tsx	Historical and forecast visualisation
analytics.ts	ZIP-level aggregations and derived calculations
types.ts	Shared data contracts
dashboard.css	Layout, typography and responsive styles

summary.json supplies the latest snapshot and national history. History and forecast JSON files are partitioned by state and loaded as needed. The browser has no live PostgreSQL connection. Missing growth stays blank; chart gaps are not silently interpolated.

Behaviour and verification

Overview and Explorer use geographic filters. ZIP Detail and Forecasts use the selected ZIP independently of those filters. Reset view restores the defaults; page reload also clears the prior in-memory selections. History ranges include one year, five years and all history. Forecast intervals can switch between 80% and 95%.

Release verification included source reconciliation, TypeScript/build checks, view navigation, a California filter, ZIP histories, forecasts and reset behaviour. The live route was opened through the portfolio CTA without signing in. CSV-generation content was independently checked, including leading-zero ZIP handling and spreadsheet formula safeguards; an actual browser download-save event was not captured during verification.

Local web setup

```
git clone https://github.com/yuvrajriyar/portfolio-website.git
cd portfolio-website
npm install
npm run dev
```

Open the local server's /projects/estateflow/dashboard route. To check a production build, run npm run build, then npm run start. The committed static snapshot works without database credentials.

13 | First-run pipeline operating guide

These commands assume a macOS/Linux shell and run from the repository root. Windows users should translate shell-specific copy and continuation syntax appropriately. Power BI Desktop itself requires Windows.

Prerequisites

Install Git, Python 3.12, uv and Docker with Compose. Start Docker before invoking Compose. Use an available host port. The project uses PostgreSQL 17-alpine with a persistent Docker volume and a health check.

```
git clone https://github.com/yuvrajriyar/EstateFlow.git
cd EstateFlow
cp .env.example .env
```

Edit .env locally. The required names are POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD and POSTGRES_PORT. The example database/user are estateflow and estateflow_user; the default port is 5432. Choose your own password. Never commit .env or publish credentials.

```
uv sync --locked
docker compose up -d --wait postgres
docker compose ps
uv run python src/estateflow/run_pipeline.py
```

Look for a healthy database, reconciled loads and zero reported quality violations, ending with EstateFlow pipeline completed successfully. The committed files reproduce the reviewed vintage. Running the source updater is a separate decision and may retrieve a newer revised vintage.

Forecast and quality checks

```
uv run python src/estateflow/run_forecast.py --publish-to-db
docker compose exec -T postgres psql -U estateflow_user \
  -d estateflow -v ON_ERROR_STOP=1 -f - \
  < sql/quality/check_zip_market_forecasts.sql
uv run python -m unittest discover -s tests -v
```

Substitute the configured user/database in the psql command if you changed them. Forecast publication requires database source mode so observed and forecast vintages agree. The script saves forecast_evaluation.csv and zip_market_forecasts.csv under data/processed/forecasts/; these outputs are ignored by Git.

If no database is available, uv run python src/estateflow/run_forecast.py --source raw reproduces the experimental workflow from committed raw files. It does not publish a PostgreSQL forecast table.

Record actual results and skipped tests. Do not report the August counts unless the run truly used that snapshot.

14 | Refresh, export and release workflow

A. Update the analytical source

Start with a clean understanding of local Git changes. Keep a recoverable previous snapshot. Run the updater only when intentionally refreshing, then rebuild observed and forecast outputs.

```
git status --short
git pull --ff-only
docker compose up -d --wait postgres
uv run python src/estateflow/update_sources.py &&
uv run python src/estateflow/run_pipeline.py &&
uv run python src/estateflow/run_forecast.py --publish-to-db
```

If any quality step fails, stop. Preserve the error and do not refresh or publish consumers. Inspect the new manifest, actual coverage counts, forecast choices and interval coverage. Update docs/current-data.md from this run rather than carrying forward old figures.

B. Refresh and package Power BI

Connect Desktop to the rebuilt database, refresh all imported tables, review five pages, test navigation and filters, and select a forecast-eligible ZIP. Save with intended defaults. Capture fresh screenshots. Export a new self-contained PBIX and create a distinct release version rather than silently changing an old labelled snapshot.

C. Reproduce the August web export

The current exporter is deliberately pinned to August, its hashes and its reconciliation totals. It expects a source directory containing zhvi.csv.gz and zori.csv. The following example assumes sibling EstateFlow and portfolio-website directories. It copies public source files into a named staging directory, not credentials.

```
# Run from portfolio-website; sibling EstateFlow has been rebuilt.
mkdir -p export-input
cp ../EstateFlow/data/raw/Zip_zhvi_uc_sfrcondo_tier_0.33_0.67_sm_sa_month.csv.gz \
  export-input/zhvi.csv.gz
cp ../EstateFlow/data/raw/Zip_zori_uc_sfrcondomfr_sm_month.csv \
  export-input/zori.csv
python3 scripts/export-estateflow.py \
  --sources export-input \
  --forecasts ../EstateFlow/data/processed/forecasts/zip_market_forecasts.csv \
  --evaluation ../EstateFlow/data/processed/forecasts/forecast_evaluation.csv \
  --output public/data/estateflow
```

Keep export-input out of commits. This export writes public JSON files, so preserve the old public snapshot first. A different source vintage should fail the present pinned assertions. Do not simply remove them. For a future release, intentionally update the expected date, hashes, source commit, totals and displayed evaluation content after validating the new snapshot.

D. Build, publish and verify

Build the portfolio, compare headline metrics, inspect state history and forecast shards, and test the five views. Commit the intended source/JSON changes and deploy through the established Vercel project. Verify the public route and portfolio CTA after deployment, not merely the build log. A source refresh alone does not update the website or the downloadable PBIX.

15 | Troubleshooting and safe recovery

Symptom	Checks and next action
Connection timeout	Check Docker health, configured host/port and the network between Desktop and database. A healthy Mac container alone is insufficient evidence of Windows connectivity.
Password failure after editing .env	Existing Docker volume can retain the original database credentials. Do not delete the volume to fix authentication without a backup and an intentional recovery plan.
Port 5432 already used	Identify the existing listener; use another authorised port and align Python/Power BI settings.
Shared-memory error during joins	Inspect Compose shm_size: the release specifies 1gb. Recreate the service to apply configuration, retaining its data volume.
Schema/table already exists NOTICE	Normally safe on repeated runs. Distinguish NOTICE from ERROR and inspect the exit status.
Source months differ	Updater refuses the pair. Keep the last valid snapshot and wait for aligned source publication or investigate the source URLs.
Pipeline gate raises an exception	Stop release work; inspect the failing layer, grain, counts and source vintage. Do not disable the check to get a green run.
Forecast charts blank in Desktop	Select exactly one eligible ZIP; confirm the forecast table was published and imported.
Desktop buttons seem inactive	In editing mode use Ctrl-click. Check the button action and target if navigation still fails.
Windows path too long	Use a short extraction path and avoid nested package copies. The project previously required launcher/update path fixes.
Web detail remains in loading/error state	Check the deployed state JSON path and that summary, history and forecast files belong to the same release.
Web exporter rejects a fresh vintage	Expected for the August-pinned exporter. Perform a deliberate version migration, not assertion removal.

Safe diagnostic commands

```
docker compose ps
docker compose logs --tail=100 postgres
uv run python src/estateflow/run_pipeline.py --skip-loads
git status --short
```

Do not use docker compose down -v as routine troubleshooting: it deletes the persistent project database volume. Do not run the schema-dropping integration tests against your project database. Local downloads and cloned reports are independent copies; deleting or editing them does not update the original public release.

16 | Development journey and design decisions

This is an evidence-based retrospective, not a reconstructed diary with invented dates. Early September work established profiling, wide-to-long transformation, staging and joined marts. Subsequent work added exact-year comparisons, coverage indicators, Power BI views and the forecast experiment. The October release moved from locally usable software to independently accessible public artefacts.

Decisions that shaped the project

Decision	Reason and trade-off
ZIP-month core grain	Preserves geographic detail; coverage is uneven and aggregates need careful interpretation
Matched inner join	Makes rent/value comparisons refer to the same ZIP-month; intentionally loses unmatched source coverage
Exact-year join	Avoids treating twelve available observations as twelve calendar months; leaves more honest blanks
Medians and visible coverage	Reduces sensitivity to extreme values; does not eliminate composition effects or make ZIPs population-representative
Separate forecasting	Prevents experimental estimates from being mistaken for observations; adds a separate operating step
Strong flat baseline	Requires a trend model to show useful improvement; most released model choices remain flat
Imported PBIX release	Lets Desktop visitors explore without database setup; retains authoring controls and Windows dependency
Independent public web app	Removes visitor licence/account barriers; requires maintaining another presentation implementation
Fixed static snapshot	Simple, credential-free hosting; freshness requires an explicit release workflow

Problems encountered and lessons learnt

The conversation records database/provider timeouts despite a healthy container, Windows path-length errors, inactive-looking Desktop buttons and saved filters reopening in unwanted states. These illustrate different categories of problem: network reachability, packaging, editor interaction and application defaults. Resolving one category does not prove the others are correct.

Power BI public hosting proved unsuitable under the available account arrangement. A trial or a Microsoft 365 family subscription did not establish the required public-publishing capability. The project retained a downloadable PBIX and implemented a genuine browser dashboard instead. This account history is a project recollection, not a universal statement of current Microsoft licensing policy.

The strongest release lesson is to verify the whole path. Correct SQL, a successful build, a refreshed Desktop screenshot and public visitor access are different acceptance checks. The final release required all of them, with honest qualifications about what each proved.

17 | AI-assisted work, ownership and interview explanation

AI assistance supported implementation, debugging, report authoring, documentation and verification. The author remained responsible for running the local PostgreSQL pipeline, refreshing Power BI Desktop, reviewing the pages and deciding what to publish. The record does not justify claiming a measured time-saving percentage for AI assistance.

A defensible explanation

Problem: public home-value and rent files differed in coverage and format, making comparison inconsistent.

Method: use Python to standardise ZIP-month observations; SQL to join and calculate metrics; tests and quality gates to verify grain and calculations; separate observed reporting from forecast experiments; publish accessible web and Power BI experiences.

Human verification: the author rebuilt the August database, confirmed zero quality violations, published forecasts and reviewed all five Desktop pages. Source reconciliation and live browser checks supported the web release.

Outcome: 462,410 matched records across 8,424 historical ZIPs, a latest snapshot of 8,421 ZIPs, and a public dashboard alongside a reusable source project and downloadable report. These are scale and deliverable measures, not proven business revenue or user adoption.

Short project introduction

"I built EstateFlow to turn Zillow home-value and rent data into a reliable, usable market-comparison tool. The pipeline standardises more than 462,000 matched ZIP-month records in PostgreSQL, verifies the grain and calculations, and feeds five-view web and Power BI dashboards. I also built a separate forecasting experiment with chronological validation, keeping its assumptions and uncertainty visible."

Questions worth preparing for

- Why inner join, and what coverage do you lose?
- Why use the global latest month rather than each ZIP's latest observation?
- Why can median gross rent-to-value differ from the ratio of median rent and median home value?
- How do you stop a failed refresh from becoming a released snapshot?
- Why did most forecast combinations retain the flat baseline?
- What did you verify yourself, and where did AI assist?
- What would change before calling this a production service?

The strongest answer is specific and candid. Distinguish code-level controls, author's local verification and planned hardening. Do not claim investment-grade forecasting or enterprise production readiness.

18 | Maintenance, future ideas and release checklist

Priority maintenance

When Zillow publishes a new aligned vintage, retain old provenance, rerun the pipeline, compare coverage and revised history, rerun forecast evaluation, refresh Power BI and deliberately migrate the pinned web export. Inspect visitor access after deployment. No recurring refresh job is configured by this handbook.

Future ideas, not delivered features

Priority	Idea	Evidence needed before release
High	Atomic approved-snapshot publication	Consumers cannot see mixed/failed refreshes
High	Configurable vintage-aware web exporter	Manifest-driven dates/hashes and reconciliation tests
High	Prospective forecast tracking	Store issued estimates; evaluate when target observations arrive
Medium	Broader automated UI coverage	Navigation, filters, downloads and responsive checks in CI
Medium	Stronger pipeline exceptions	Critical checks remain active regardless of Python optimisation
Medium	FRED/Census context	Compatible geographic/time grain and documented join limitations
Medium	Larger-file distribution strategy	Reproducible source retrieval without repeated large Git binaries
Later	More forecast candidates	Outperform baselines on proper chronological and prospective evaluation

FRED/ACS variables could add context, but do not automatically establish causal explanations. A public live database/API would introduce security, cost and operational responsibilities that the present static release avoids.

Release checklist

1. Record dates separately; update handbook provenance and screenshots for the new vintage.
2. Confirm source identity, latest-month agreement and uncompressed hashes.
3. Complete observed loads and all SQL gates; save actual counts.
4. Review all six forecast evaluations and run forecast SQL checks.
5. Run unit tests; identify skipped integration tests; use only a disposable integration database.
6. Refresh and review all five Power BI pages, navigation, single-ZIP charts and intended defaults.
7. Reconcile web metrics and static files; build and test interactions and exports.
8. Review staged Git changes for secrets, caches and accidental source staging copies.
9. Publish versioned artefacts and deploy the portfolio.
10. Check the public CTA, route and data loading without requiring an account.

19 | Provenance and document maintenance

This edition was assembled on 1 October 2026 Pacific time, with verification extending into 2 October UTC. It describes the completed October portfolio release with August observations.

Release anchors

Artefact	Recorded identity
Published raw source commit	0ad3beb719aac81c7ebe5969d83225eb7ff63aa0
Web release commit	26d3ba63193d6f8e4c1d28c16bc725c69cbd1d15
EstateFlow web-documentation commit	a0d90f0fd028dbed8aa34ce20811bff3201a1ebc
Power BI release	v1.0.0; October 2026 publication

PBIX SHA-256: ef79447f257286eabb99378f6bf239fceb36dae34fd9feed33bf61d4ab0d85f5

ZHVI uncompressed CSV SHA-256:
9dcd2793d97e5f60727bfc10563acdebe8461e48f8af83e09c333512a00521d9

ZORI CSV SHA-256: a60963f429dbacf97e4371a8e4f9146322a09073d76031fc1fe477ca65183933

Primary implementation references

- EstateFlow README, docs/current-data.md, docs/web-dashboard.md and docs/forecast-methodology.md.
- src/estateflow/update_sources.py, run_pipeline.py, transform_zori.py, transform_zhvi.py, forecasting.py and run_forecast.py.
- sql/staging/, sql/intermediate/, sql/marts/ and sql/quality/.
- tests/test_quality_gates_postgres.py and .github/workflows/tests.yml.
- powerbi/EstateFlow_PowerBI_Analytics/README.md and the editable PBIP.
- Portfolio scripts/export-estateflow.py and app/projects/estateflow/dashboard/README.md.
- Author-supplied successful August pipeline and forecast logs, Git publication output and Desktop acceptance confirmations.
- Public route verification after the Vercel release.

How to maintain this handbook

Keep the Markdown as the editable source. Update measured results only from a completed run. Preserve historical experiment dates, revise released access links if necessary, and regenerate the PDF after changes. If code and prose disagree, inspect the implementation and actual run evidence rather than choosing the more flattering description.

Known legacy wording: older repository notes still describe July experiment results or pending Power BI Service publication. That does not reverse the shipped web-dashboard status. Power BI Service itself is not the public web-hosting mechanism. This handbook makes those distinctions explicit rather than silently rewriting historical records.

End of release edition 1.0.